Lab 4 — Vector + Raster Processing (GeoPandas + rasterio)

Open-Source Geovisualization • GEOG 8005 / ESCI 6000 • Feb 09, 2026

Overview

In this lab you will practice a professional workflow for working with both vector and raster data in Python. You will load datasets, verify CRS, perform basic processing, create summary outputs, and export analysis-ready files that can support your course project.

What you will produce (portfolio-friendly)

- A clean vector layer clipped to your study area (GeoPackage)
- A clipped raster (GeoTIFF) with documented CRS/resolution/nodata
- A points layer with sampled raster values (GeoPackage)
- Two quick visual checks (maps) and basic summary statistics

Prerequisites

- A working conda environment (e.g., geoviz) with: geopandas, rasterio, numpy, pandas, matplotlib
- A raster from Lab 3 (DEM, Landsat, Sentinel-2, NDVI, etc.) saved as a GeoTIFF
- A study area boundary polygon (GeoJSON/GeoPackage). If you do not have one yet, create a simple polygon in QGIS or draw one in geojson.io

```
# In terminal (example)
conda activate geoviz
python -c "import geopandas, rasterio, numpy, pandas; print('OK')"
```

Data for this lab

Use ONE of the following options:

Option A (recommended): Use your Lab 3 downloads

- Raster: your GeoTIFF (DEM or imagery-derived product)
- Vector: your study boundary polygon (GeoJSON/GPKG)
- Optional: any additional vector layer you downloaded (roads, tracts, parcels, etc.)

Option B: Use a local open dataset you already know

- Example: City/county open data portal boundary + a DEM tile from USGS
- Keep it small so processing runs quickly on your laptop

Submission (screenshots only)

Submit a single PDF (or a set of image files) containing these screenshots:

1. Successful imports + package versions (GeoPandas, rasterio)
2. Vector inspection output: CRS + total_bounds + a preview of columns/head()
3. Map preview: study boundary + clipped vector layer
4. Raster metadata output: CRS + transform + bounds + nodata + resolution
5. Raster preview plot (full raster)
6. Masked/clipped raster preview plot + nodata-aware min/mean/max
7. Points sampling output: first 5 rows showing sampled values
8. Final check: map of sampled points (colored) on top of the study boundary
9. File explorer screenshot showing your outputs/ folder contains exported files

Part 1 — Vector workflow (GeoPandas)

1.1 Read + inspect

- Load your study boundary polygon and a vector layer (e.g., roads/tracts).
- Print CRS and bounds; confirm they make sense.

```
import geopandas as gpd

study = gpd.read_file("data/raw/study_area.geojson")
vec   = gpd.read_file("data/raw/vector_layer.gpkg")  # e.g., roads, tracts

print("Study CRS:", study.crs)
print("Study bounds:", study.total_bounds)
print("Vector CRS:", vec.crs)
print("Vector bounds:", vec.total_bounds)
print(vec.head())
```

1.2 CRS workflow (detect → verify → transform)

- If CRS is missing: stop and verify from the source before setting it.
- Transform BOTH layers to a working projected CRS for your area (meters).

```
# Example: choose a local UTM zone (replace EPSG if needed)
WORK_EPSG = "EPSG:32617"  # example only
```

```
study_m = study.to_crs(WORK_EPSG)
vec_m   = vec.to_crs(WORK_EPSG)
```

1.3 Clip + basic aggregation

- Clip vector layer to study boundary.
- Compute ONE simple summary statistic (lines: length; polygons: area).
- Export the clipped layer to a GeoPackage.

```
import geopandas as gpd

vec_clip = gpd.clip(vec_m, study_m)

# Example summaries (choose relevant)
if vec_clip.geom_type.isin(["LineString","MultiLineString"]).any():
    vec_clip["len_m"] = vec_clip.length
    print("Total length (km):", vec_clip["len_m"].sum()/1000)

if vec_clip.geom_type.isin(["Polygon","MultiPolygon"]).any():
    vec_clip["area_m2"] = vec_clip.area
    print("Total area (km2):", vec_clip["area_m2"].sum()/1e6)

# Export
vec_clip.to_file("outputs/vector_clipped.gpkg", layer="vector_clipped",
driver="GPKG")
```

1.4 Quick map preview (vector)

```
import matplotlib.pyplot as plt

ax = study_m.plot(facecolor="none", edgecolor="black", linewidth=2,
figsize=(8,6))
vec_clip.plot(ax=ax, linewidth=0.8, alpha=0.8)
ax.set_title("Study area + clipped vector")
ax.set_axis_off()
plt.show()
```

Part 2 — Raster workflow (rasterio)

2.1 Open + inspect metadata

- Open your GeoTIFF and print CRS, bounds, transform, resolution, nodata, dtype.

```
import rasterio
import numpy as np

raster_path = "data/raw/raster.tif"  # your file

with rasterio.open(raster_path) as src:
    print("CRS:", src.crs)
    print("Bounds:", src.bounds)
    print("Transform:", src.transform)
    print("Resolution:", (src.transform.a, -src.transform.e))
    print("Nodata:", src.nodata)
    print("Count:", src.count, "dtype:", src.dtypes)
```

2.2 Plot a raster preview + nodata-aware stats

```
import matplotlib.pyplot as plt

with rasterio.open(raster_path) as src:
    band1 = src.read(1)
    nod = src.nodata
    data = np.where(band1 == nod, np.nan, band1) if nod is not None else band1

    print("min/mean/max:", np.nanmin(data), np.nanmean(data), np.nanmax(data))

    plt.figure(figsize=(8,6))
    plt.imshow(band1)
    plt.title("Raster preview (band 1)")
    plt.axis("off")
    plt.show()
```

2.3 Mask/clip raster by polygon

- Reproject study polygon to the raster CRS (not the other way around).
- Use rasterio.mask.mask to crop to the polygon.
- Save the clipped raster to outputs/ as a new GeoTIFF.

```
from rasterio.mask import mask

with rasterio.open(raster_path) as src:
```

```
    study_r = study.to_crs(src.crs)
    shapes = [g.__geo_interface__ for g in study_r.geometry]

    out_img, out_transform = mask(src, shapes, crop=True)
    out_meta = src.meta.copy()
    out_meta.update({
        "height": out_img.shape[1],
        "width": out_img.shape[2],
        "transform": out_transform
    })

clipped_path = "outputs/raster_clipped.tif"
with rasterio.open(clipped_path, "w", **out_meta) as dst:
    dst.write(out_img)
print("Saved:", clipped_path)
```

Part 3 — Integration: sample raster values at points

3.1 Create random points inside the study area (GeoPandas)

```
from shapely.geometry import Point
import random
import geopandas as gpd
import rasterio

with rasterio.open("outputs/raster_clipped.tif") as src:
    crs_r = src.crs

study_r = study.to_crs(crs_r)
poly = study_r.geometry.unary_union

N = 50
minx, miny, maxx, maxy = poly.bounds

pts_list = []
while len(pts_list) < N:
    x = random.uniform(minx, maxx)
    y = random.uniform(miny, maxy)
    p = Point(x, y)
    if poly.contains(p):
        pts_list.append(p)

pts = gpd.GeoDataFrame({"id": range(1, N+1)}, geometry=pts_list, crs=crs_r)
```

3.2 Sample raster at those points (rasterio)

```
import rasterio

with rasterio.open("outputs/raster_clipped.tif") as src:
    coords = [(g.x, g.y) for g in pts.geometry]
    vals = [v[0] for v in src.sample(coords)]

pts["value"] = vals
print(pts.head())
pts.to_file("outputs/points_sampled.gpkg", layer="points_sampled",
driver="GPKG")
```

3.3 Map check: points colored by sampled value

```
import matplotlib.pyplot as plt

ax = study_r.plot(facecolor="none", edgecolor="black", linewidth=2,
figsize=(8,6))
pts.plot(ax=ax, column="value", legend=True, markersize=25)
ax.set_title("Sampled points (value)")
ax.set_axis_off()
plt.show()
```

Checklist before you submit

- You verified CRS at every step (vector-vector, vector-raster)
- You handled nodata when computing raster statistics
- Your outputs/ folder contains: vector_clipped.gpkg, raster_clipped.tif, points_sampled.gpkg
- Your screenshots clearly show results (not tiny) and include file paths